Prise en main de webpack 5

Prise en main de webpack 5

> À la fin de ce guide, vous aurez construit **de zéro** un projet web complet : un code source découpé en modules JavaScript, des feuilles de style, des images, du code transpilé pour les navigateurs anciens, un serveur de développement qui recharge la page toute seule, et une version de production optimisée. Vous n'avez **rien** à connaître de Node.js ni de npm pour commencer.

Ce guide se suit **dans l'ordre, du début à la fin**. Chaque étape part de l'état laissé par la précédente. À chaque fois, vous trouverez :

- **ce qu'on veut faire** et *pourquoi* ;
- **les commandes exactes** à taper ;
- **le contenu complet** des fichiers (pas des extraits à recoller) ;
- une section **« Vérification »** qui décrit ce que vous devez obtenir. Si vous n'obtenez pas ça, ne passez pas à la suite : allez voir la section **Dépannage**, à la fin du document.

> **Note sur la documentation officielle.** La documentation de webpack (https://webpack.js.org/ (https://webpack.js.org/)) est la référence, mais elle est écrite par morceaux indépendants qui ne s'enchaînent pas toujours : une page vous fait créer un fichier qu'une autre supprime, certains exemples sont incomplets, et plusieurs guides n'ont pas été mis à jour en même temps que l'outil. **Ce document est une traversée cohérente et testée** de ce que ces pages racontent séparément. Consultez la doc officielle pour approfondir un point précis, pas pour suivre un chemin.

Partie 0 — De quoi parle-t-on ?

0.1 Le problème que webpack résout

Écrivons une petite page web « à l'ancienne ». Le code est découpé en trois fichiers pour rester lisible :

```
<script src="outils.js"></script>
<script src="panier.js"></script>
<script src="application.js"></script>
```

Cette façon de faire pose quatre problèmes très concrets :

1. **L'ordre est critique et invisible.** Si `panier.js` a besoin d'une fonction définie dans `outils.js`, il *faut* que `outils.js` soit chargé avant. Rien dans le code ne le dit : c'est l'ordre des balises `<script>` qui décide. Un jour, quelqu'un déplace une ligne et tout casse.
2. **Tout est global.** Chaque fichier déclare ses variables dans le même espace : `window`. Deux

fichiers qui utilisent tous les deux une variable `total` s'écrasent mutuellement, silencieusement.
3. **Une requête HTTP par fichier.** Dix modules, dix allers-retours réseau. Sur une connexion mobile, ça se voit.
4. **On ne peut pas utiliser les bibliothèques de l'écosystème.** Les dizaines de milliers de bibliothèques JavaScript publiées sur npm sont distribuées sous forme de **modules**, pas de balises `<script>`.

Le langage JavaScript a résolu les points 1 et 2 avec les **modules ES** (`import` / `export`), que vous connaissez déjà. Mais un navigateur qui charge cent petits modules fait cent requêtes, et ne sait pas aller chercher tout seul une bibliothèque installée depuis npm.

0.2 Ce que fait webpack

webpack est un *bundler* (un « empaqueteur »). On lui donne **un point d'entrée** — un fichier JavaScript — et il fait ceci :

1. il lit ce fichier et repère tous ses `import` ;
2. il ouvre chaque fichier importé et recommence, récursivement ;
3. il construit ainsi un **graphe de dépendances** : la carte complète de tout ce dont votre application a besoin ;
4. il assemble le tout en **un seul fichier** (ou quelques-uns), prêt à être servi au navigateur.

Ce graphe ne contient pas que du JavaScript. Grâce aux **loaders** et aux **modules d'assets**, une feuille de style, une image ou une police peuvent être importées comme n'importe quel module, et donc suivies, optimisées et renommées automatiquement.

Enfin, webpack peut **transformer** le code au passage : le traduire pour des navigateurs anciens (transpilage), le minifier, ajouter une empreinte dans les noms de fichiers pour gérer le cache, etc.

0.3 Les cinq mots de vocabulaire à retenir

Ces cinq notions structurent le fichier de configuration. Vous les rencontrerez à toutes les étapes.

Terme	En une phrase
entry	Le fichier par lequel webpack commence à explorer. Par défaut `./src/index.js`.
output	Où webpack écrit le résultat, et sous quel nom. Par défaut `./dist/main.js`.
loader	Un traducteur appliqué à un fichier **avant** qu'il entre dans le graphe (ex. : transpiler du JS moderne en JS ancien).
plugin	Un outil qui agit sur **l'ensemble** de la compilation (ex. : générer un fichier HTML, vider le dossier de sortie).
mode	`development` (rapide, lisible, pour travailler) ou `production` (lent, minifié, pour livrer).

> **Loader ou plugin ?** Un *loader* transforme **un fichier à la fois**, et on le déclare dans `module.rules`. Un *plugin* travaille sur **toute la compilation**, et on le déclare dans `plugins`. Retenez ça, c'est la question la plus fréquente.

Partie 1 — Installer et comprendre Node.js et npm

webpack est un programme écrit en JavaScript. Pour l'exécuter **sur votre machine** (et non dans un navigateur), il faut Node.js.

1.1 Installer Node.js

Rendez-vous sur https://nodejs.org/ (https://nodejs.org/) et installez la version **LTS** (*Long Term Support*, la version stable maintenue sur la durée — la 24.x à la rédaction de ce guide). Prenez l'installateur correspondant à votre système, et laissez toutes les options par défaut.

> **Version minimale.** Les outils utilisés ici exigent au moins **Node 22.18**. Si vous avez une version plus ancienne installée, mettez-la à jour, sinon vous rencontrerez des erreurs incompréhensibles à l'installation des paquets.

Ouvrez ensuite **un terminal** :

- **Windows** : PowerShell, ou le terminal intégré de VS Code (menu *Terminal → Nouveau terminal*) ;
- **Linux / macOS** : votre terminal habituel.

Vérifiez l'installation :

```
node --version
npm --version
```

Vérification. Vous devez voir deux numéros de version, par exemple `v24.14.0` et `11.6.2`. Si le terminal répond « commande introuvable », l'installation a échoué ou le terminal était déjà ouvert avant l'installation : fermez-le et rouvrez-en un.

1.2 Node.js, npm, npx : qui fait quoi ?

Trois noms, trois rôles bien distincts :

- **Node.js** est un *environnement d'exécution* : il exécute du JavaScript en dehors d'un navigateur. `node mon-script.js` lance un fichier, exactement comme `python mon-script.py`.
- **npm** (*node package manager*) est le *gestionnaire de paquets* installé en même temps que Node. Il télécharge des bibliothèques depuis le dépôt public https://www.npmjs.com/ (https://www.npmjs.com/) et note dans votre projet ce que vous avez installé.
- **npx** est un *lanceur*. Il exécute un programme installé dans votre projet sans que vous ayez à retrouver son chemin exact.

> La documentation de webpack mentionne aussi `yarn` et `pnpm` : ce sont des gestionnaires de paquets **concurrents** de npm, qui font le même travail avec une syntaxe différente. **N'utilisez que `npm`** dans ce guide, et ne mélangez jamais deux gestionnaires dans un même projet.

1.3 Créer le projet

Créez un dossier de travail, placez-vous dedans, et initialisez le projet :

```
mkdir tp-webpack
cd tp-webpack
npm init -y
```

`npm init` crée un fichier **`package.json`** : c'est la **carte d'identité du projet**. Le `-y` accepte toutes les réponses par défaut au lieu de vous poser une dizaine de questions.

Ouvrez le dossier `tp-webpack` dans votre éditeur (`code .` avec VS Code) et regardez `package.json` :

```
{
  "name": "tp-webpack",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Les champs qui vont nous servir :

- **`scripts`** : des raccourcis de commandes. Nous y mettrons `build` et `start`.
- **`dependencies`** et **`devDependencies`** : la liste des bibliothèques du projet (elles apparaîtront à la première installation).
- **`type`** : la syntaxe de modules JavaScript utilisée par le projet. npm met `"commonjs"` par défaut, ce qui ne nous convient pas : **remplacez tout de suite `"commonjs"` par `"module"`**, puis enregistrez. Si la ligne est absente (versions de npm antérieures à la 11), ajoutez `"type": "module"` vous-même. La raison est expliquée en 3.1.

> **Ne sautez pas cette modification.** Avec `"type": "commonjs"`, webpack lit vos fichiers comme de l'ancien JavaScript et refuse les `import` que vous écrirez en Partie 2, avec l'erreur `'import' and 'export' may appear only with 'sourceType: module'`.

1.4 Installer webpack

```
npm install --save-dev webpack webpack-cli
```

Décortiquons :

- `webpack` est le moteur ; `webpack-cli` est l'interface en ligne de commande qui permet de l'appeler depuis un terminal. **Les deux sont nécessaires.**
- `--save-dev` (abrégeable en `-D`) range ces paquets dans **`devDependencies`** : ce sont des *outils de développement*. Ils servent à **fabriquer** l'application, mais ne partent pas avec elle chez l'utilisateur. Une bibliothèque dont votre code a besoin **à l'exécution** (par exemple une bibliothèque de graphiques que vous `import`ez) s'installerait sans ce drapeau, et atterrirait dans `dependencies`.

Trois choses sont apparues dans votre dossier :

Élément	Rôle	À versionner (git) ?
`node_modules/`	Le code téléchargé des bibliothèques et de **toutes** leurs propres dépendances. Des milliers de fichiers.	**Non, jamais.**
`package-lock.json`	La liste des versions **exactes** installées, pour que tout le monde ait rigoureusement le même projet.	**Oui.**
`devDependencies` dans `package.json`	La liste de ce que *vous* avez demandé, avec les versions acceptables.	**Oui.**

> **Pourquoi ne pas versionner `node_modules/` ?** Parce que `package.json` + `package-lock.json` suffisent à le reconstruire à l'identique : sur une machine neuve, un simple `npm install` (sans argument) le régénère.

Créez tout de suite un fichier **`.gitignore`** à la racine du projet, avec comme contenu :

```
node_modules/
dist/
```

Vérification. `package.json` contient maintenant une section `devDependencies` avec `webpack` et `webpack-cli`.

Partie 2 — Votre premier bundle

2.1 Écrire le code source

La convention (et le comportement par défaut de webpack) est de mettre le **code source** dans `src/` et le **résultat compilé** dans `dist/` (pour *distribution*). Vous n'écrivez **jamais** à la main dans `dist/` : ce dossier est jetable et régénérable.

Créez le dossier `src` et deux fichiers.

`src/salutation.js` — un module qui exporte une fonction :

```
export function saluer (nom) {
  return `Bonjour ${nom} !`
}
```

`src/index.js` — le point d'entrée, qui importe le module :

```
import { saluer } from './salutation.js'

const titre = document.createElement('h1')
titre.textContent = saluer('webpack')
document.body.append(titre)
```

`index.html` — à la **racine** du projet (pas dans `src/`, pas dans `dist/`) :

```html
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Mon projet webpack</title>
  </head>
  <body>
    <script src="./dist/main.js"></script>
  </body>
</html>
```

> **Attention.** La documentation officielle place ce fichier dans `dist/`. C'est une mauvaise idée : `dist/` est un dossier **généré**, et nous activerons plus loin une option qui le vide à chaque compilation — votre HTML écrit à la main disparaîtrait. Nous le mettons à la racine pour l'instant, et en Partie 6 c'est webpack qui le produira pour nous.

Votre projet ressemble à ceci :

```
tp-webpack/
├── .gitignore
├── index.html
├── package.json
├── package-lock.json
├── node_modules/
└── src/
    ├── index.js
    └── salutation.js
```

2.2 Compiler

```
npx webpack
```

`npx` va chercher le webpack installé dans `node_modules/` de **ce** projet et l'exécute. Sans aucune configuration, webpack applique ses valeurs par défaut : il part de `./src/index.js` et écrit `./dist/main.js`.

Vous obtenez un compte rendu, suivi d'un avertissement :

```
WARNING in configuration
The 'mode' option has not been set, webpack will fallback to 'production' for this
value.
```

C'est normal : nous n'avons pas encore dit à webpack dans quel mode travailler. Nous réglons ça tout de suite.

Vérification. Un dossier `dist/` contenant `main.js` est apparu. Ouvrez `index.html` dans un navigateur (double-clic) : la page affiche **« Bonjour webpack ! »**. Ouvrez aussi `dist/main.js` : vous y verrez le code des **deux** fichiers sources réunis, compressé sur une seule ligne.

Partie 3 — Le fichier de configuration

Se contenter des valeurs par défaut ne mène pas loin. Toute la suite du guide consiste à enrichir **un fichier de configuration**.

3.1 Passer le projet en modules ES

Avant d'écrire la configuration, une formalité. Node.js sait lire deux syntaxes de modules : l'ancienne (`require` / `module.exports` , dite *CommonJS*) et la moderne (`import` / `export` , dite *ESM*, celle que vous utilisez déjà dans `src/`). Pour n'en utiliser **qu'une seule** dans tout le projet, on le déclare dans `package.json` : c'est le `"type": "module"` que vous avez mis en 1.3. On en profite pour faire le ménage dans le reste du fichier.

Remplacez le contenu de **`package.json`** par :

```
{
  "name": "tp-webpack",
  "version": "1.0.0",
  "description": "",
  "private": true,
  "type": "module",
  "scripts": {
    "build": "webpack"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "webpack": "^5.111.1",
    "webpack-cli": "^7.2.3"
  }
}
```

> **Gardez vos propres numéros de version.** Ceux de `devDependencies` peuvent différer légèrement chez vous : conservez ceux qui figurent dans **votre** fichier. En revanche, la section elle-même ne doit **pas** disparaître.

Ce qui a changé :

- **`"type": "module"`** (déjà en place depuis 1.3) : tous les fichiers `.js` du projet sont des modules ES. webpack s'en sert pour savoir comment lire vos fichiers de `src/` , et Node.js pour lire la configuration de webpack : c'est ce qui nous permettra d'y écrire `import` .
- **`"private": true`** : empêche une publication accidentelle du projet sur npm.
- **`scripts.build`** : notre raccourci. `npm run build` exécutera `webpack` .
- La ligne `"main"` a disparu (elle ne sert que pour une bibliothèque publiée), et le script `test` par défaut aussi.
- `devDependencies` n'a pas changé : c'est npm qui maintient cette section, vous n'avez pas à y toucher.

> **Pourquoi insister sur `devDependencies` ?** Si la section disparaît, rien ne casse tout de suite : webpack est toujours dans `node_modules/`. Mais au prochain `npm install --save-dev …`, npm aligne `node_modules/` sur `package.json` et **supprime** tout paquet qui n'y est plus listé. webpack disparaît alors, et `npm run build` échoue deux parties plus loin, sans lien apparent avec cette étape.

> **Attention.** Dans un projet `"type": "module"`, écrire `const path = require('path')` provoque l'erreur `require is not defined in ES module scope`. C'est l'erreur n° 1 quand on recopie un tutoriel ancien. Voir la section **Dépannage**.

3.2 Écrire la configuration

Créez `webpack.config.js` à la racine (webpack cherche ce nom automatiquement) :

```
import path from 'node:path'

export default {
  mode: 'development',
  entry: './src/index.js',
  output: {
    filename: 'main.js',
    path: path.resolve(import.meta.dirname, 'dist')
  }
}
```

Ligne par ligne :

- `import path from 'node:path'` : on importe le module `path`, **fourni par Node.js** (le préfixe `node:` le signale), qui sait manipuler des chemins de fichiers correctement sur tous les systèmes.
- `export default { ... }` : on exporte un objet de configuration. C'est cet objet que webpack va lire.
- `mode: 'development'` : compilation rapide, code lisible, pas de minification. L'avertissement de la Partie 2 disparaît.
- `entry` : le point de départ du graphe de dépendances.
- `output.filename` : le nom du fichier produit.
- `output.path` : **le dossier de sortie, en chemin absolu**. webpack l'exige, pour éviter toute ambiguïté sur l'endroit où il va écrire. `import.meta.dirname` est le dossier qui contient le fichier de configuration ; `path.resolve(...)` le combine avec `'dist'` pour produire un chemin absolu valable sous Windows comme sous Linux.

> Vous verrez souvent dans les tutoriels la formule `path.resolve(__dirname, 'dist')`. `__dirname` est l'équivalent CommonJS de `import.meta.dirname` ; il n'existe pas dans un module ES, d'où les quatre lignes d'incantation (`fileURLToPath(import.meta.url)` …) que la doc officielle place en tête de ses exemples. `import.meta.dirname` rend tout cela inutile.

3.3 Compiler avec le script npm

```
npm run build
```

`npm run build` exécute la commande inscrite dans `scripts.build`. Deux avantages sur `npx webpack` : la commande ne change jamais, même quand la vraie commande sous-jacente se complique (ce qui arrivera en Partie 8) ; et elle est documentée dans le projet, donc utilisable par n'importe qui.

Vérification. La compilation se fait sans avertissement. `dist/main.js` fait maintenant quelques kilo-octets et son contenu est **lisible** (mode développement). La page affiche toujours « Bonjour webpack ! ».

Partie 4 — Les assets : CSS, images, polices

Jusqu'ici, webpack n'a manipulé que du JavaScript. Son vrai intérêt commence maintenant : **faire entrer les autres ressources dans le graphe de dépendances**.

L'idée est de ne plus déclarer une image ou une feuille de style « quelque part dans le HTML », mais de l'`import`er depuis le module qui en a besoin. Le bénéfice est immédiat : si vous supprimez un composant, ses styles et ses images disparaissent du bundle **automatiquement**, parce que plus personne ne les importe. Plus de fichiers morts.

4.1 Les feuilles de style

Créez `src/style.css` :

```css
body {
  margin: 0;
  padding: 2rem;
  font-family: system-ui, sans-serif;
  background: #1d1f27;
  color: #f2f2f2;
}

h1 {
  color: #7ee0c0;
}
```

Importez-le **en première ligne** de `src/index.js` :

```
import './style.css'
import { saluer } from './salutation.js'

const titre = document.createElement('h1')
titre.textContent = saluer('webpack')
document.body.append(titre)
```

> Remarquez la forme de l'import : pas de `{ }`, pas de nom. On n'a besoin d'aucune valeur du fichier CSS, on veut juste dire à webpack : *« ce module dépend de cette feuille de style »*.

Compilez :

```
npm run build
```

Le compte rendu mentionne un **nouvel élément** : `asset main.css`. webpack 5 sait traiter le CSS **nativement**, sans installer quoi que ce soit, et il produit un fichier `.css` séparé du JavaScript.

Il reste à le charger dans la page. Ajoutez la ligne `<link>` dans **`index.html`** :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Mon projet webpack</title>
    <link rel="stylesheet" href="./dist/main.css">
  </head>
  <body>
    <script src="./dist/main.js"></script>
  </body>
</html>
```

Vérification. `dist/` contient maintenant `main.js` **et** `main.css`. La page a un fond sombre et un titre vert d'eau.

> **Pourquoi la plupart des tutoriels installent-ils `style-loader` et `css-loader` ?** Parce que le support natif du CSS est récent. Il est piloté par l'option `experiments.css`, qui vaut `'auto'` depuis webpack 5.109 : **activée par défaut**, sauf si vous déclarez vous-même une règle avec un loader pour les `.css`, auquel cas c'est votre règle qui gagne. Ce support est encore officiellement étiqueté « expérimental » et deviendra le comportement standard de webpack 6.
>
> La chaîne `['style-loader', 'css-loader']` reste parfaitement valable et vous la croiserez partout : `css-loader` traduit le CSS en module JavaScript, puis `style-loader` l'injecte dans une balise `<style>` **à l'exécution**. Différence visible : avec cette chaîne il n'y a pas de fichier `.css` produit, tout est dans le JS. C'est pratique en développement, moins en production (la page s'affiche brièvement sans style). Les deux approches fonctionnent ; **n'en mélangez pas deux dans le même projet**.

4.2 Les images

Placez une image dans `src/`, par exemple **`src/logo.png`** (n'importe quelle image fera l'affaire ; renommez-la `logo.png`).

Modifiez **`src/index.js`** :

```
import './style.css'
import logo from './logo.png'
import { saluer } from './salutation.js'

const titre = document.createElement('h1')
titre.textContent = saluer('webpack')
document.body.append(titre)

const image = new Image()
image.src = logo
image.alt = 'Logo'
document.body.append(image)
```

Ici l'import a un nom : `logo` contiendra **l'URL finale de l'image**, celle que webpack aura choisie après l'avoir copiée dans `dist/`. C'est tout l'intérêt : vous ne codez jamais le chemin de sortie en dur, webpack s'en charge.

Compilez. **Échec** :

```
ERROR in ./src/logo.png
Module parse failed: Unexpected character '?' (1:0)
You may need an appropriate loader to handle this file type...
```

Ce message est **le plus fréquent de tous** avec webpack. Il dit : *« j'ai rencontré un fichier dans le graphe, j'ai essayé de le lire comme du JavaScript, et ce n'en est pas. Dis-moi quoi en faire. »*

On le lui dit avec une **règle de module**. Remplacez `webpack.config.js` par :

```
import path from 'node:path'

export default {
  mode: 'development',
  entry: './src/index.js',
  output: {
    filename: 'main.js',
    path: path.resolve(import.meta.dirname, 'dist')
  },
  module: {
    rules: [
      {
        test: /\.(png|jpe?g|gif|svg|webp)$/i,
        type: 'asset/resource',
        generator: {
          filename: 'images/[name].[contenthash][ext]'
        }
      },
      {
        test: /\.(woff2?|ttf|otf|eot)$/i,
        type: 'asset/resource',
        generator: {
          filename: 'fonts/[name].[contenthash][ext]'
        }
      }
    ]
  }
}
```

Une règle se lit toujours de la même façon :

- `test` : une expression régulière appliquée au **chemin du fichier**. Le `$` ancre la fin du nom, le `i` rend la casse indifférente (`.PNG` passe aussi).
- `type: 'asset/resource'` : le traitement à appliquer. Ici : *« copie ce fichier dans le dossier de sortie et donne-moi son URL »*.
- `generator.filename` : comment nommer la copie. `[name]` reprend le nom d'origine, `[ext]` l'extension, et `[contenthash]` insère une empreinte calculée sur le contenu du fichier. Ce dernier point est important : si l'image change, son nom change, donc le navigateur est *obligé* de retélécharger ; si elle ne change pas, il garde sa copie en cache indéfiniment.

Les quatre types d'assets disponibles, pour information :

Type	Effet
`asset/resource`	Copie le fichier à part et renvoie son URL. **Le cas normal.**
`asset/inline`	Encode le fichier en base64 **dans** le bundle. Utile pour de toutes petites icônes.
`asset/source`	Renvoie le contenu brut du fichier sous forme de chaîne (ex. : un `.txt`).
`asset`	Choisit automatiquement entre les deux premiers selon la taille (seuil : 8 Kio).

Compilez de nouveau.

Vérification. Plus d'erreur. `dist/images/` contient votre image sous un nom du genre `logo.419b1acd21e8e451e52d.png` , et la page l'affiche.

> **Et les images référencées depuis le CSS ?** Une déclaration `background: url('./logo.png')` dans `src/style.css` est traitée automatiquement : webpack réécrit l'URL vers le fichier copié. Vous n'avez rien à configurer.

4.3 Les polices

La deuxième règle que nous avons ajoutée couvre les polices. Le mécanisme est identique : déposez un `.woff2` dans `src/` , déclarez-le en CSS, et webpack le copiera dans `dist/fonts/` en réécrivant l'URL :

```
@font-face {
  font-family: 'MaPolice';
  src: url('./ma-police.woff2') format('woff2');
}
```

Partie 5 — Le transpilage avec Babel

5.1 Pourquoi transpiler

Vous écrivez du JavaScript moderne : champs privés de classe (`#valeur`), opérateur de coalescence (`??`), chaînage optionnel (`?.`)… Les navigateurs récents comprennent tout cela. Les navigateurs plus anciens, non — et « plus ancien » peut vouloir dire « le navigateur du téléphone de votre utilisateur, jamais mis à jour ».

Transpiler, c'est traduire automatiquement votre code moderne en code équivalent compris par les navigateurs que vous visez. L'outil standard s'appelle **Babel**. On le branche dans webpack via un **loader** : `babel-loader`.

C'est exactement le rôle décrit en Partie 0 : un loader transforme un fichier **avant** qu'il n'entre dans le graphe.

5.2 Un module à transpiler

Créez **`src/compteur.js`**, qui utilise un champ privé (syntaxe moderne) :

```
export class Compteur {
  #valeur = 0

  incrementer () {
    this.#valeur += 1
    return this.#valeur
  }

  get valeur () {
    return this.#valeur
  }
}
```

Et complétez **`src/index.js`** :

```
import './style.css'
import logo from './logo.png'
import { saluer } from './salutation.js'
import { Compteur } from './compteur.js'

const titre = document.createElement('h1')
titre.textContent = saluer('webpack')
document.body.append(titre)

const image = new Image()
image.src = logo
image.alt = 'Logo'
document.body.append(image)

const compteur = new Compteur()
const bouton = document.createElement('button')
bouton.textContent = 'Clics : 0'
bouton.addEventListener('click', () => {
  bouton.textContent = `Clics : ${compteur.incrementer()}`
})
document.body.append(bouton)
```

5.3 Installer et configurer Babel

```
npm install --save-dev babel-loader @babel/core @babel/preset-env
```

Trois paquets, trois rôles :

- **`@babel/core`** : le moteur de traduction.

- **`@babel/preset-env`** : le « preset » qui décide **quelles** transformations appliquer. Sans lui, Babel ne fait rien. Sa force : il déduit tout seul les transformations nécessaires à partir de la liste des navigateurs que vous visez.
- **`babel-loader`** : la colle entre webpack et Babel.

Ajoutez la règle **en premier** dans `module.rules` de `webpack.config.js` :

```
      {
        test: /\.js$/i,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: ['@babel/preset-env']
          }
        }
      },
```

Deux points méritent une explication :

- **`exclude: /node_modules/`** est indispensable. Sans lui, Babel retraduirait les milliers de fichiers de vos dépendances — déjà compilées — et votre build passerait de deux secondes à plusieurs minutes.
- **`use`** désigne le ou les loaders à appliquer. Quand il y en a plusieurs, on donne un tableau, et **webpack les applique de droite à gauche** (le dernier du tableau s'exécute en premier). Cette règle de lecture surprend tout le monde la première fois.

Enfin, indiquez **quels navigateurs vous visez**. Cela se fait dans `package.json`, avec une clé `browserslist` — un format standard, compris par Babel mais aussi par d'autres outils :

```
  "browserslist": [
    "> 0.5%",
    "last 2 versions",
    "not dead"
  ]
```

Ce qui se lit : *« les navigateurs utilisés par plus de 0,5 % des internautes, plus les deux dernières versions de chaque navigateur, en excluant ceux qui ne sont plus maintenus. »*

Votre `package.json` ressemble maintenant à ceci :

```
{
  "name": "tp-webpack",
  "version": "1.0.0",
  "description": "",
  "private": true,
  "type": "module",
  "scripts": {
    "build": "webpack"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "@babel/core": "^8.0.6",
    "@babel/preset-env": "^8.0.6",
    "babel-loader": "^10.1.1",
    "webpack": "^5.111.1",
    "webpack-cli": "^7.2.3"
  },
  "browserslist": [
    "> 0.5%",
    "last 2 versions",
    "not dead"
  ]
}
```

Compilez.

Vérification. La compilation réussit et la page affiche un bouton qui compte les clics.

5.4 Constater le transpilage

Tel quel, vous ne voyez rien : avec des cibles modernes, Babel n'a presque rien à traduire. Faites l'expérience suivante.

Remplacez temporairement la valeur de `browserslist` par `["chrome 60"]` (un Chrome de 2017), recompilez, puis cherchez la chaîne `#valeur` dans `dist/main.js` : **elle a disparu**. À la place, Babel a réécrit la classe en utilisant une `WeakMap` pour simuler le champ privé, qui n'existait pas à l'époque.

Remettez la valeur d'origine et recompilez : `#valeur` est de retour tel quel.

C'est tout l'intérêt de `preset-env` : **il ne transforme que ce qui est nécessaire**. Élargir la cible aux vieux navigateurs alourdit le bundle ; la restreindre le rend plus léger et plus rapide. C'est un arbitrage de projet, pas un réglage technique.

> **Note.** Babel ne touche pas aux `import` / `export` : webpack lui indique qu'il sait les gérer lui-même, et il en a besoin pour construire son graphe et éliminer le code mort. Ne vous étonnez donc pas de voir des `export` subsister dans les fichiers intermédiaires.

Partie 6 — Laisser webpack produire le HTML

6.1 Le problème

Notre `index.html` contient deux lignes écrites à la main :

```
<link rel="stylesheet" href="./dist/main.css">
<script src="./dist/main.js"></script>
```

Elles sont fragiles. Dès que nous ajouterons une empreinte au nom des fichiers (`main.4a12b4fc….css` , en Partie 8), ces chemins deviendront faux à chaque compilation. Et il est hors de question de les corriger à la main.

La solution est un **plugin** : `html-webpack-plugin` . Il prend un **gabarit** HTML, y injecte tout seul les balises vers les fichiers réellement produits, et écrit le résultat dans `dist/` .

6.2 Mise en place

```
npm install --save-dev html-webpack-plugin
```

Déplacez `index.html` de la racine vers `src/index.html` , et **supprimez-en** les balises `<link>` et `<script>` — c'est le plugin qui s'en occupe désormais :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Mon projet webpack</title>
  </head>
  <body>
  </body>
</html>
```

Dans `webpack.config.js` , importez le plugin en haut du fichier et ajoutez une section `plugins` :

```
import path from 'node:path'
import HtmlWebpackPlugin from 'html-webpack-plugin'

export default {
  mode: 'development',
  entry: './src/index.js',
  output: {
    filename: 'main.js',
    path: path.resolve(import.meta.dirname, 'dist')
  },
  plugins: [
    new HtmlWebpackPlugin({
      template: './src/index.html'
    })
  ],
  module: {
    rules: [
      // … vos règles, inchangées
    ]
  }
}
```

Notez la forme : un plugin s'**instancie** avec `new`, contrairement à un loader qu'on désigne par son nom. C'est la conséquence directe de ce que nous disions en Partie 0 : un plugin est un objet qui s'accroche au cycle de vie de la compilation, pas une fonction appliquée à un fichier.

Compilez, puis ouvrez **`dist/index.html`** (et non plus celui de la racine) :

```
<!DOCTYPE html>
<html lang="fr">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Mon projet webpack</title>
  <script defer src="main.js"></script><link href="main.css" rel="stylesheet"></head>
  <body>
  </body>
</html>
```

Les deux balises ont été injectées, avec les bons chemins — et `defer` sur le script, pour qu'il s'exécute après l'analyse du document.

Vérification. Ouvrir `dist/index.html` dans le navigateur donne exactement la même page qu'avant : titre, image, bouton, styles.

> À partir de maintenant, **tout le contenu de `dist/` est généré**. Vous n'y écrivez plus jamais rien, et vous pouvez le supprimer entièrement à tout moment : `npm run build` le reconstruit.

Partie 7 — Le serveur de développement et le rechargement à chaud

7.1 Pourquoi

Le cycle actuel est pénible : modifier un fichier, taper `npm run build`, attendre, recharger le navigateur. Vous allez le faire plusieurs centaines de fois par séance.

`webpack-dev-server` supprime ces trois étapes. Il lance un petit serveur web local qui :

- surveille vos fichiers sources et **recompile automatiquement** à chaque enregistrement ;
- garde les fichiers produits **en mémoire vive** (rien n'est écrit sur le disque : c'est plus rapide, et `dist/` n'est pas pollué) ;
- pousse les modifications dans la page ouverte via une connexion permanente avec le navigateur.

Ce dernier point mérite une distinction que beaucoup de monde confond :

- le **rechargement automatique** (*live reload*) recharge la page entière, comme si vous appuyiez sur F5 ;
- le **rechargement à chaud** (*Hot Module Replacement*, HMR) remplace **uniquement le module modifié**, dans la page déjà ouverte, **sans la recharger**. Tout ce que vous aviez à l'écran — un formulaire à moitié rempli, un menu ouvert, un compteur à 17 — est conservé.

7.2 Mise en place

```
npm install --save-dev webpack-dev-server
```

Ajoutez deux sections à `webpack.config.js`, juste avant `plugins` :

```
  devtool: 'eval-source-map',
  devServer: {
    port: 8080,
    open: true,
    hot: true,
    watchFiles: ['src/**/*.html']
  },
```

Et ajoutez un script `start` dans `package.json` :

```
  "scripts": {
    "start": "webpack serve",
    "build": "webpack"
  }
```

Les options :

- **`port: 8080`** : l'adresse sera http://localhost:8080/ (http://localhost:8080/). Changez ce numéro si le port est déjà pris.
- **`open: true`** : ouvre le navigateur automatiquement au démarrage.
- **`hot: true`** : active le rechargement à chaud (c'est déjà la valeur par défaut, on l'écrit pour la lisibilité).
- **`watchFiles: ['src/**/*.html']`** : par défaut, le serveur ne surveille que les fichiers présents dans le graphe de dépendances. Le gabarit HTML n'en fait pas partie ; cette ligne demande explicitement de le surveiller aussi.
- **`devtool: 'eval-source-map'`** : les ***source maps***. Sans elles, une erreur JavaScript vous

renvoie à la ligne 3412 du bundle, ce qui ne vous apprend rien. Avec elles, la console du navigateur affiche **le nom de votre fichier source et le bon numéro de ligne**, et le débogueur vous montre votre code d'origine. `eval-source-map` est un bon compromis en développement : précis, et rapide à régénérer.

7.3 Lancer et essayer

```
npm start
```

Le terminal affiche :

```
<i> [webpack-dev-server] Project is running at:
<i> [webpack-dev-server] Loopback: http://localhost:8080/
webpack 5.111.1 compiled successfully in 814 ms
```

Le navigateur s'ouvre sur votre page. **Laissez ce terminal ouvert** : le serveur tourne dedans. Pour l'arrêter, `Ctrl+C`.

Faites maintenant les deux expériences suivantes, dans cet ordre. Elles sont le cœur de cette partie.

Expérience 1 — le CSS, en HMR. Dans la page, cliquez trois fois sur le bouton (il affiche « Clics : 3 »). Sans toucher au navigateur, modifiez la couleur de fond dans `src/style.css` et enregistrez. Regardez la page :

- la couleur change **immédiatement** ;
- le compteur affiche **toujours « Clics : 3 »**.

La page n'a **pas** été rechargée : seul le module CSS a été remplacé à chaud. C'est ça, le HMR.

Expérience 2 — le JavaScript, en rechargement complet. Modifiez maintenant `src/salutation.js` (remplacez « Bonjour » par « Salut ») et enregistrez :

- le titre change ;
- mais le compteur est **revenu à « Clics : 0 »**.

La page entière a été rechargée. Pourquoi cette différence ? Parce que remplacer un module JavaScript à chaud n'a rien d'automatique : webpack ne peut pas deviner ce qu'il faut faire de l'ancien état (les objets déjà créés, les écouteurs déjà posés). Un module doit **explicitement déclarer** qu'il sait se faire remplacer, avec `import.meta.webpackHot.accept()`. Tant qu'aucun module ne le fait, le serveur se rabat sur un rechargement complet de la page — ce qui est un comportement sûr et déjà très confortable.

> En pratique, ce sont les frameworks (React, Vue…) qui fournissent cette logique de remplacement pour leurs composants, via leurs propres outils. Dans un projet en JavaScript pur, on se contente très bien du HMR sur le CSS et du rechargement automatique sur le JS.

Vérification. Les deux expériences donnent bien les résultats décrits. Une erreur de syntaxe dans un fichier source s'affiche en surimpression dans le navigateur, et disparaît dès que vous la corrigez.

Partie 8 — Développement et production

8.1 Deux besoins opposés

Vous avez maintenant une configuration confortable pour **travailler**. Mais elle est inadaptée à la **livraison** :

	Développement	**Production**
Vitesse de compilation	**critique** (des centaines de fois par jour)	sans importance (une fois avant la livraison)
Taille du résultat	sans importance	**critique** (temps de chargement)
Lisibilité du code	**critique** (débogage)	inutile, voire indésirable
Source maps	complètes et rapides	fichiers séparés, pour ne pas alourdir
Nom des fichiers	stables	avec empreinte, pour le cache navigateur

On ne peut pas satisfaire les deux avec un seul fichier. La solution retenue par webpack : **une configuration commune, et deux variantes** qui la complètent.

8.2 Découper la configuration

```
npm install --save-dev webpack-merge
```

`webpack-merge` fusionne intelligemment deux objets de configuration : là où une simple fusion d'objets écraserait un tableau de règles, `merge` les concatène.

Supprimez `webpack.config.js` et créez trois fichiers à la place.

`webpack.common.js` — tout ce qui ne dépend pas de l'environnement :

```
import path from 'node:path'
import HtmlWebpackPlugin from 'html-webpack-plugin'

export default {
  entry: './src/index.js',
  output: {
    path: path.resolve(import.meta.dirname, 'dist'),
    clean: true
  },
  plugins: [
    new HtmlWebpackPlugin({
      template: './src/index.html'
    })
  ],
  module: {
    rules: [
      {
        test: /\.js$/i,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: ['@babel/preset-env']
          }
        }
      },
      {
        test: /\.(png|jpe?g|gif|svg|webp)$/i,
        type: 'asset/resource',
        generator: {
          filename: 'images/[name].[contenthash][ext]'
        }
      },
      {
        test: /\.(woff2?|ttf|otf|eot)$/i,
        type: 'asset/resource',
        generator: {
          filename: 'fonts/[name].[contenthash][ext]'
        }
      }
    ]
  }
}
```

Nouveauté : **`clean: true`** vide `dist/` avant chaque compilation. Sans cette option, les fichiers des compilations précédentes s'accumulent — et avec des noms contenant une empreinte, on se retrouve vite avec des dizaines de bundles obsolètes. C'est aussi la raison pour laquelle nous n'avons jamais mis de fichier écrit à la main dans `dist/`.

`webpack.dev.js` :

```
import { merge } from 'webpack-merge'
import commun from './webpack.common.js'

export default merge(commun, {
  mode: 'development',
  devtool: 'eval-source-map',
  output: {
    filename: 'js/[name].js',
    cssFilename: 'css/[name].css'
  },
  devServer: {
    port: 8080,
    open: true,
    hot: true,
    watchFiles: ['src/**/*.html']
  }
})
```

`webpack.prod.js` :

```
import { merge } from 'webpack-merge'
import commun from './webpack.common.js'

export default merge(commun, {
  mode: 'production',
  devtool: 'source-map',
  output: {
    filename: 'js/[name].[contenthash].js',
    cssFilename: 'css/[name].[contenthash].css'
  }
})
```

> **Attention.** Dans un projet `"type": "module"`, l'extension `.js` est **obligatoire** dans les imports relatifs : `'./webpack.common.js'`, jamais `'./webpack.common'`. C'est la règle des modules ES, et c'est une source d'erreur classique.

Enfin, les deux scripts de `package.json` doivent désigner la bonne configuration :

```
  "scripts": {
    "start": "webpack serve --config webpack.dev.js",
    "build": "webpack --config webpack.prod.js"
  }
```

Voilà pourquoi il fallait passer par `npm run` plutôt que par `npx webpack` : la commande réelle s'est compliquée, mais **`npm start`** et **`npm run build`** **n'ont pas changé**.

8.3 Ce que fait `mode: 'production'`

Il ne s'agit pas d'un simple drapeau : ce réglage active en bloc une série d'optimisations.

- **Minification** du JavaScript et du CSS : espaces et commentaires supprimés, noms de variables locales raccourcis.
- **Élimination du code mort** (*tree shaking*) : une fonction exportée que personne n'importe n'est

pas incluse dans le bundle. C'est possible uniquement grâce aux modules ES, dont les dépendances sont analysables statiquement.
- `process.env.NODE_ENV` est fixé à `'production'`, ce dont de nombreuses bibliothèques se servent pour désactiver leurs vérifications de développement.

Côté source maps, `devtool: 'source-map'` produit des fichiers `.map` **séparés**. Le navigateur ne les télécharge que si quelqu'un ouvre les outils de développement : vos utilisateurs ne paient rien, et vous gardez la possibilité de déboguer un problème en production.

8.4 Compiler pour la production

```
npm run build
```

Le résultat :

```
dist/
├── css/
│   ├── main.4a12b4fca8ec2fb9e30c.css
│   └── main.4a12b4fca8ec2fb9e30c.css.map
├── images/
│   └── logo.419b1acd21e8e451e52d.png
├── index.html
└── js/
    ├── main.0bbadcf0b6a2b46cd21f.js
    └── main.0bbadcf0b6a2b46cd21f.js.map
```

Et `dist/index.html`, minifié, pointe sur les bons fichiers :

```
<script defer src=js/main.0bbadcf0b6a2b46cd21f.js></script><link href=css/
main.4a12b4fca8ec2fb9e30c.css rel=stylesheet>
```

C'est la force de la combinaison `[contenthash]` + `html-webpack-plugin` : les noms changent à chaque modification réelle du contenu, et le HTML suit **tout seul**. Vous pouvez donc configurer votre serveur pour mettre ces fichiers en cache pendant un an sans jamais servir une version périmée.

Vérification. `dist/` ne contient **que** les fichiers ci-dessus (l'option `clean` a fait le ménage). Ouvrez `dist/index.html` dans un navigateur : la page fonctionne, image et bouton compris. Comparez la taille de `js/main.[hash].js` avec celle obtenue en mode développement : elle a été divisée par plusieurs.

> **Le dossier `dist/` est ce que vous déployez.** C'est lui, et lui seul, que vous copiez sur un serveur web. Ni `src/`, ni `node_modules/`, ni les fichiers de configuration.

Récapitulatif

Arborescence finale

```
tp-webpack/
├── .gitignore
├── package.json
├── package-lock.json
├── webpack.common.js
├── webpack.dev.js
├── webpack.prod.js
├── node_modules/          (généré par npm, jamais versionné)
├── dist/                  (généré par webpack, jamais versionné, jamais édité)
└── src/
    ├── index.html         (le gabarit)
    ├── index.js           (le point d'entrée)
    ├── salutation.js
    ├── compteur.js
    ├── style.css
    └── logo.png
```

Les deux seules commandes à connaître

Commande	Effet
`npm start`	Lance le serveur de développement sur http://localhost:8080/, avec recompilation et rechargement automatiques. `Ctrl+C` pour arrêter.
`npm run build`	Produit la version de production optimisée dans `dist/`.

Et sur une machine où vous venez de récupérer le projet (sans `node_modules/`) :

Commande	Effet
`npm install`	Réinstalle toutes les dépendances à partir de `package.json` et `package-lock.json`.

Ce que vous devez savoir expliquer

Si vous ne deviez retenir que cela :

1. webpack part d'un **point d'entrée**, suit tous les `import`, et produit un **bundle**.
2. Un **loader** transforme un fichier à son entrée dans le graphe ; il se déclare dans `module.rules` et s'applique **de droite à gauche** quand il y en a plusieurs.
3. Un **plugin** agit sur toute la compilation ; il se déclare dans `plugins` et s'instancie avec `new`.
4. Le `mode` bascule en bloc des dizaines d'optimisations : c'est le premier réglage à vérifier.
5. Les **source maps** (`devtool`) sont ce qui rend le bundle déboguable ; sans elles, vous travaillez à l'aveugle.
6. `[contenthash]` dans les noms de fichiers + `html-webpack-plugin` = gestion du cache navigateur résolue, sans intervention manuelle.

Dépannage

Les erreurs les plus fréquentes, et ce qu'elles veulent dire.

`'webpack' n'est pas reconnu` / **`webpack: command not found`** webpack est installé *dans le projet*, pas globalement sur la machine. Utilisez `npm run build`, `npm start`, ou `npx webpack`. N'installez **pas** webpack en global (`-g`) : vous vous exposeriez à des conflits de version entre pro-

jets. Si l'erreur apparaît alors que la compilation fonctionnait auparavant, regardez `package.json` : webpack et webpack-cli ont sans doute disparu de `devDependencies` (typiquement en recopiant un `package.json` complet), et un `npm install` ultérieur les a retirés de `node_modules/`. Réinstallez-les : `npm install --save-dev webpack webpack-cli`.

`require is not defined in ES module scope, you can use import instead` Votre `package.json` contient `"type": "module"` et vous avez recopié un exemple ancien qui utilise `require` / `module.exports`. Traduisez-le : `const x = require('y')` devient `import x from 'y'`, et `module.exports = {…}` devient `export default {…}`.

`Module parse failed: 'import' and 'export' may appear only with 'sourceType: module'` — `File was parsed as module type 'javascript/dynamic'` Votre `package.json` contient `"type": "commonjs"` (ajouté d'office par `npm init` depuis npm 11) : webpack lit alors vos fichiers `.js` comme du CommonJS, où `import` est interdit. Remplacez-le par `"type": "module"` (voir 1.3) et relancez la compilation.

`Cannot find module '…/webpack.common'` — `Did you mean to import "./webpack.common.js"?` Dans un module ES, l'extension `.js` est obligatoire dans les imports relatifs. Ajoutez-la.

`Module parse failed: Unexpected character … You may need an appropriate loader to handle this file type` webpack a rencontré un fichier qu'il ne sait pas traiter (une image, une police, un `.scss`…) et a essayé de le lire comme du JavaScript. Il manque une règle dans `module.rules` dont le `test` couvre cette extension.

`Module not found: Error: Can't resolve './style.css'` Le chemin de l'import est faux : faute de frappe, mauvaise casse (attention, Linux distingue les majuscules, pas Windows), ou fichier placé ailleurs. Les chemins relatifs commencent par `./` ou `../` ; sans ce préfixe, webpack cherche dans `node_modules/`.

`Error: listen EADDRINUSE: address already in use :::8080` Le port 8080 est déjà occupé, généralement par un serveur de développement que vous n'avez pas arrêté. Fermez-le (`Ctrl+C` dans son terminal) ou changez `port` dans `webpack.dev.js`.

Le serveur tourne, mais mes modifications n'apparaissent pas. Trois causes possibles. (1) Vous regardez `dist/index.html` ouvert en `file://` au lieu de http://localhost:8080/ (http://localhost:8080/) — le serveur ne sert pas ce fichier-là, il travaille en mémoire. (2) Vous avez modifié le gabarit HTML sans avoir l'option `watchFiles`. (3) Votre éditeur utilise une écriture « sécurisée » qui trompe la surveillance de fichiers : désactivez *safe write* dans les préférences (JetBrains, Sublime Text).

`WARNING in configuration – The 'mode' option has not been set` Vous avez lancé `webpack` sans `--config`, ou sur une configuration où `mode` est absent. En Partie 8, la commande doit désigner explicitement `webpack.dev.js` ou `webpack.prod.js`.

J'ouvre `dist/index.html` et la page est blanche. Ouvrez la console du navigateur (F12). Neuf fois sur dix, il s'agit d'un chemin cassé ou d'une exception JavaScript — que les source maps vous permettront de localiser dans votre code source.

Le build est devenu très lent. Vérifiez que la règle `babel-loader` contient bien `exclude: /node_modules/`.

J'ai lancé `npm install` sans `--save-dev` par erreur. Le paquet est allé dans `dependencies`

au lieu de `devDependencies`. Sans gravité pour faire fonctionner le projet, mais corrigez-le : `npm uninstall <paquet>` puis réinstallez avec `--save-dev`.

Pour aller plus loin

Vous maîtrisez maintenant le socle. Voici les prolongements naturels, par ordre de rentabilité.

Sass. Un préprocesseur CSS qui ajoute variables, imbrication et fonctions. C'est un excellent exercice de chaîne de loaders :

```
npm install --save-dev sass sass-loader
```

```
      {
        test: /\.s[ac]ss$/i,
        type: 'css',
        use: ['sass-loader']
      },
```

`sass-loader` traduit le Sass en CSS, et `type: 'css'` confie le résultat au support CSS natif de webpack. Documentation : https://webpack.js.org/loaders/sass-loader/ (https://webpack.js.org/loaders/sass-loader/)

Un linter. Un outil qui signale les erreurs et les incohérences de style **pendant** que vous écrivez. `standard` impose un style unique sans configuration ; ESLint est plus paramétrable. Voir https://standardjs.com/ (https://standardjs.com/).

Le découpage de bundle (*code splitting*). Pour ne pas charger d'un bloc une application qui grossit : https://webpack.js.org/guides/code-splitting/ (https://webpack.js.org/guides/code-splitting/)

Plusieurs points d'entrée. Pour une application multi-pages : la clé `entry` accepte un objet, et `[name]` dans `output.filename` reprend alors le nom de chaque entrée.

Le cache de compilation. L'option `cache: { type: 'filesystem' }` accélère nettement les recompilations d'un gros projet.

Documentation de référence

- Concepts : https://webpack.js.org/concepts/ (https://webpack.js.org/concepts/)
- Configuration complète : https://webpack.js.org/configuration/ (https://webpack.js.org/configuration/)
- Modules d'assets : https://webpack.js.org/guides/asset-management/ (https://webpack.js.org/guides/asset-management/)
- `babel-loader` : https://webpack.js.org/loaders/babel-loader/ (https://webpack.js.org/loaders/babel-loader/)
- Serveur de développement : https://webpack.js.org/configuration/dev-server/ (https://webpack.js.org/configuration/dev-server/)
- Production : https://webpack.js.org/guides/production/ (https://webpack.js.org/guides/production/)
- Browserslist : https://browsersl.ist/ (https://browsersl.ist/)

> **Un conseil pour lire cette documentation.** Prenez-la comme un dictionnaire, pas comme un roman : allez-y avec une question précise (« que fait l'option `devtool` ? »), lisez la page correspondante, revenez. Suivre plusieurs guides à la chaîne vous fera écrire des configurations contradictoires.

©DESBENOIT Brett - IUT d'Aix-Marseille